

Analyse des problèmes v5 → v6

Fichier : analyse_problemes_v5_v6.md **Dossier :** wiki racine **Branche :** master (wiki)
Auteur : hprzeta · **MAJ :** 2026-06-11

Résumé exécutif

	v5 (Voie B)	v6 (scan_arb + STEP=0.010)
Détection	Z_vect_correct numpy batch	scan_arb.c Z_double C pur
STEP	adaptatif /3 (~0.22 à T=100k)	0.010 fixe (gap-safe GUE)
Segmentation	1/√t approx	N(T) par bisection
Zéros T=100k	135 997	138 069
Manquants	2 072	0
Turing		COMPLET

1. Problème — STEP adaptatif /3 insuffisant

Cause mathématique

v5 utilisait $\text{STEP} = (t)/3 = 2/(3 \cdot \ln(t/2))$. À $t = 100\,000$:

$$\delta(100\,000) = \frac{2\pi}{\ln(100\,000/2\pi)} \approx 0.66 \Rightarrow \text{STEP}_{v5} \approx 0.22$$

La loi de Wigner (GUE) prédit la distribution des gaps normalisés $s = \delta_n/\delta(t)$:

$$p_{\text{GUE}}(s) = \frac{\pi}{2} s e^{-\pi s^2/4}$$

Le gap minimal mesuré sur les 10 000 premiers zéros est $\delta_{\min} \approx 0.019$ (à $t \approx 66\,678$). Le STEP v5 (≈ 0.22) est donc $\times 11$ trop grand — il manquait tous les pairs de zéros séparés par moins de 0.22.

Quantification de la perte

$$\text{manquants} = N_{\text{total}} \cdot P(\delta_n < \text{STEP}) \approx 138\,069 \times F_{\text{GUE}}\left(\frac{0.22}{0.66}\right) \approx 2\,072$$

Résultat mesuré : 2 072 manquants — pire que v1 (356 avec paliers 0.1/0.05/0.02).

Solution v6

STEP fixe 0.010 = gap_min/2 = 0.019/2 :

$$\text{STEP}_{\text{safe}} = 0.010 \leq \frac{\delta_{\min}}{2} \approx 0.0095$$

Résultat : **0 manquant** sur 138 069 zéros. Contre-intuitivement, STEP=0.010 est plus rapide que /3 car il évite les faux positifs à retraiter (moins d'intervalles sans zéro).

2. Problème — scan Python (numpy) trop lent

Cause

v5 utilisait `Z_vect_correct` (détection numpy vectorisée) avec un callback Python pour chaque point. Le STEP=0.010 implique $100\,000/0.010 = 10^7$ évaluations $Z(t)$ pour un scan complet — soit ~17 min de scan pur si chaque évaluation coûte 0.1 ms.

De plus, le passage Python→C (ctypes) pour chaque appel Illinois représente un overhead de ~30 μ s, non négligeable à 8+ itérations par zéro.

Solution v6 — scan_arb.c

`scan_arb.c` réécrit le scan entier en C avec `arb_fpwrap_cdouuble_hardy_z` (libflint) : - Aucune allocation heap (double IEEE 754 pur, registres CPU) - Loop C pure : 0.77ms/appel vs 1.2ms Python loop overhead inclus - Passage du tableau de brackets directement en mémoire partagée

```
/* scan_arb.c - boucle principale */
int scan_zeros_arb(double t_min, double t_max, double step,
                  double *out_a, double *out_b,
                  double *out_fa, double *out_fb, int n_max) {
    complex double z_prev, z_cur;
    arb_fpwrap_cdouuble_hardy_z(&z_prev, t_min, 0);
    int n = 0;
    for (double t = t_min + step; t <= t_max && n < n_max; t += step) {
        arb_fpwrap_cdouuble_hardy_z(&z_cur, t, 0);
        if (creal(z_prev) * creal(z_cur) < 0) {
            out_a[n]=t-step; out_b[n]=t;
            out_fa[n]=creal(z_prev); out_fb[n]=creal(z_cur);
            n++;
        }
        z_prev = z_cur;
    }
    return n;
}
```

Gain mesuré : $\times 5.7$ sur la phase détection (0.2 % du temps CPU total en v6 vs ~6 % en v5).

3. Problème — Segmentation approximée

Cause

v5 segmentait les intervalles de t par portions de longueur $\propto 1/\sqrt{t}$ (pour équilibrer la charge). L'approximation sous-estimait systématiquement le nombre de zéros dans les segments à grand t , causant un déséquilibre de ~15 % entre workers.

Solution v6 — N(T) par bisection

Utilisation de la formule exacte de Riemann-von Mangoldt pour calculer les bornes de segments :

$$N(T) = \frac{T}{2\pi} \ln \frac{T}{2\pi e} + 1 + S(T)$$

Bisection sur $N(T)$ pour trouver T_k tel que $N(T_k) = k \cdot N(T_{\max})/W$. Déséquilibre résiduel : <3 % (vs 15 % en v5).

4. Résultats mesurés — T=100 000

Métrique	v5	v6
Zéros trouvés	135 997	138 069
Manquants	2 072	0
Turing-Backlund		COMPLET
Durée réelle	~1h58 (estimé)	1h53
LMFDB 19/20 < 1e-10		
Illinois_C pur > 50 %	~85 %	83 %

5. Bottleneck résiduel — illinois_C = 83 %

L'affinage Illinois à 170 bits (libmpfr, PREC=170) représente 83 % du temps total.

$$N_{\text{termes}}(t) = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor \Rightarrow N(100\,000) = 126 \text{ termes}$$

Chaque terme requiert ≈ 1.1 ms (mpfr_cos + mpfr_log à 170 bits). Illinois = 2–5 évaluations Z par zéro :

$$T_{\text{illinois}} \approx 138\,069 \times 3.5 \times 126 \times 1.1 \text{ ms}/4 \approx 1.67 \text{ h}$$

Prochain levier (v7) : N_termes adaptatif dans `illinois_mpfr.c` — schéma 2-phases ou réduction PREC 170→100 bits.

6. Questions ouvertes

1. **LMFDB #20 (t 77.14)** : erreur résiduelle 8e-10 1e-10 — limitation structurelle RS à 2 termes ?
 2. **PREC=100 bits** : suffisant pour tolérance 1e-10 sur t [300, 100 000] ?
 3. **STEP=0.010 à T>100k** : gap_min décroît-il assez vite pour nécessiter STEP<0.010 ?
-

Voir aussi

- [[analyse_problemes_v4_1_v6_synthese]] — vue globale v4.1→v6
- [[Formules_zeta]] §19 (STEP/GUE) · §20 (coût scan) · §21 (N_termes RS)
- [[Bibliotheques]] §12 — tous les runs T=100k

analyse_problemes_v5_v6.md · wiki racine · branche master · hprzeta · MAJ 2026-06-11 · 108 lignes